Rust Raft ledger

Assessment of the current five-crate workspace and its 17 design RFCs. The application is a backend ledger, so usability was assessed through its gRPC contract, Rust client and operator CLI.

Release judgment

Continue with integration and fault testing. I would defer a production rollout until a returning voter can regain service reliably under load, operators can verify redundancy, and the recovery journal has been tested in an independent failure domain.

Area	Assessment	Evidence behind the judgment
Architecture	Sound foundation	One ordered decision per request, a deterministic engine and explicit boundaries between consensus, the working state and the recovery copy.
Distributed correctness	Strong local evidence	Financial invariants and protected reads survive the tested failures. Permanent voter replacement and large-backlog recovery have important gaps.
App usability	Clear payment contract	Same-ID retries and typed rejections are usable. Setup, live health inspection and the reservation inquiry workflow need work.
Performance	Useful local baseline	Batching buys throughput. Tail latency varies with load and recovery. The journal and a single leader bound each ledger group.

Most consequential result

All **216 default tests passed**. In one of three release load trials, a returning voter could not acquire the shared journal lock within two seconds. Two fresh repeats passed the restart point and reconciled the public workload. This is an availability finding. The failed run ended before its final balance reconciliation.

How to read the evidence

“Observed” identifies this review’s executions. “Recorded” identifies checked-in measurements. “Source finding” identifies a code path with an explained consequence. Failure frequencies, deployment costs and an acceptable service-level target remain open.

Architecture and data ownership

One ledger and asset form one consensus group. All accounts touched by a compound payment share its ordering and commit boundary.

Component	What it owns	Review judgment
ledger-core	Checked integer balances, funding, holds, capture and original decisions.	Deterministic evaluation has a narrow state-reading interface. The same rules work in memory and against storage.
ledger-api	Public Protobuf schema, normalization and Rust seed routing.	Clients can integrate without carrying RocksDB or consensus dependencies.
ledger-replica	Raft driver, durable log, rebuildable RocksDB projection, cache and snapshots.	One writer orders changes. Most integration complexity sits at the boundary between log, projection and snapshot.
ledger-recovery	Independent POSIX journal, segment validation and fencing.	A separate authority enables recovery of acknowledged history after voter-disk loss. Its availability is on the request path.
ledger-node	Peer and business listeners, admission, owner thread, protection worker and CLI.	Bounded queues and separate peer lanes protect foreground work. Some synchronous disk operations still share the Raft owner.

What a successful payment reply means



The public handler normalizes the request. The leader groups waiting commands, replicates the entry and persists the commit boundary. The evaluator stages effects in order. One projection write publishes the affected rows, receipt and applied index. The response waits for application and independent journal coverage in the relevant leadership term.

crates/ledger-node/src/client_service.rs:119 · ledger-replica/src/replica.rs:469 · ledger-replica/src/projection/delta.rs:83 · ledger-node/src/runtime.rs:407 (crate paths under crates/).

The cost of the design

One leader and one journal publication order make per-ledger capacity finite. The WAL-disabled projection is defensible because restart reconstructs it from a seal and committed history. Request lookup and audit indexes grow with lifetime use. Physical Raft reclamation moves covered command bytes to a pinned copy, while receipt and projection storage continue to grow.

ARCHITECTURE.md:7 · docs/rfcs/0014-immutable-ledger-history.md:15 · docs/rfcs/0017-raft-log-reclamation.md · crates/ledger-replica/src/raft_store/reclaim.rs:360

Financial and distributed correctness

The tests exercise three promises a backend depends on. Accepted postings balance, retries preserve the original decision, and successful current reads reflect a protected ledger state.

Property	Implementation and evidence
Atomic money movement	Balanced lines and checked counters are staged before publication. A failure on the last account leaves earlier accounts untouched. The independent wallet model exercised 3 seeds × 4,096 commands and checked counters after every step.
Retry identity	The first command and its outcome survive subsequent state changes. Same ID and payload returns the original result. A changed payload produces an ID conflict. Business rejections are retained alongside accepted decisions.
Consensus and apply	The owner persists Ready/LightReady state before releasing dependent Raft messages, checks the durable commit index before applying entries, and stops after persistence/apply failure. A persisted uncommitted suffix is excluded from replay.
Current reads	A read uses a term-bound quorum ReadIndex, waits for apply, samples one local view and then waits for journal coverage of that sampled index. Multi-account reads are tested across concurrent writes on the embedded interface.
Recovery integrity	Journal publication uses an expected head, immutable bytes, readback and directory sync. Fencing prevents the old generation from continuing protected service. Reclamation pins and verifies the independent copy before switching the local log.

What happens when a dependency disappears

One voter stops. The other two can serve while the journal is available. **Quorum is lost.** New commits and current public reads cannot complete. **The journal is offline.** Some local application can progress, but protected success is withheld. A client that timed out must reuse the original request ID and payload.

The local tests include a lost leader, a partially captured waterfall restored after all voter directories were removed, a fenced old cluster, and damaged recovery bytes. Deterministic scenarios and the wallet oracle provide different evidence. A systematic cross-host check of response histories under overlapping faults is still outstanding.

crates/ledger-core/tests/mixed_payments.rs:48 · ledger-core/tests/ledger.rs:184 · ledger-replica/tests/replication.rs:289 · ledger-node/src/reads.rs:172 · ledger-node/tests/network/recovery.rs:308,440,481,535 · ledger-recovery/src/journal.rs:254. Paths under crates/.

Reliability findings

These findings affect service availability or the ability to restore redundancy. The lock timeout was observed. The other mechanisms were traced in the current code.

High priority / A returning voter can fail to open its journal

During a release workload, the follower reopened its local database and then Journal::open returned “recovery journal lock timed out.” The open path reads the head under the same exclusive lock used for publication. Acquisition polls with a two-second limit. The CLI propagates this startup error. A fresh trial joined after 361 ms including reopen. Lock-holder timing and the recurrence rate were unmeasured. Capture wait/hold times by operation and qualify startup under sustained writes.

crates/ledger-node/tests/network/api/load.rs:543 · ledger-node/src/main.rs:148 · ledger-recovery/src/journal.rs:69,145,176,254 (under crates/). Attached release-first and release-repeat logs.

High priority / A blank replacement disk cannot rejoin online

Fresh initialization requires an empty journal. Restore requires a fenced generation. The two survivors can keep serving, but the supported recovery path to three fresh voters interrupts protected service. Time that interval through the first successful payment. If uninterrupted service is required, implement and validate a safe replacement procedure that preserves votes and quorum.

docs/rfcs/0006-raft-replication.md:13 · crates/ledger-node/src/main.rs:157 · crates/ledger-replica/src/recovery.rs:168 · README.md:157

Medium priority / Raft can build an append larger than the wire limit

The locked raft-rs version can concatenate successive size-limited reads into one append during catch-up. The node enables this behavior with a 2 MiB read limit, then rejects combined messages over 4 MiB and reports the peer unreachable. Large retained backlogs can waste work and slow progress. Verify emitted frame sizes and follower convergence with realistic, large entries. Client-command microbatching is a separate layer.

crates/ledger-replica/src/replica.rs:295 · crates/ledger-node/src/transport.rs:37,85,492 · crates/ledger-node/src/runtime.rs:137 · raft-rs 0.7.0 src/raft.rs:760,780,830,1808

Medium priority / A checkpoint can delay Raft heartbeats

The owner flushes all six projection families, captures the local checkpoint and opens it for validation before a worker takes over hashing. A recorded warm run paused apply for 28 ms. The owner ticks every 50 ms and the base election interval is 10 ticks. Slow-device pauses can disrupt leadership. Measure maximum owner pauses, elections and payment p99 during the same capture.

crates/ledger-node/src/checkpoints.rs:148 · crates/ledger-replica/src/projection/checkpoint.rs:179 · docs/rfcs/0012-incremental-snapshots.md:37 · Raft paper §5.6, Raft paper

Application design and usability

Three practical journeys expose the difference between a good payment protocol and a service that a backend team can operate on its own.

Connect and make a first payment

The schema has six mutations and three protected lookups. Canonical decimal strings preserve 128-bit values across languages. The SDK uses configured TLS seeds and one overall deadline. Explicit handling for partial capture and typed rejections make important financial choices visible.

The setup example needs three configurations and multiple certificate files. I ran its journal initialization verb in a new directory. It failed because the parent recovery directory was missing, then succeeded after creating that parent. **--help** exited 1 and printed one usage line. The README's public-client example looks up request 17 on a blank ledger. A runnable local setup should create two accounts, post, repeat the ID, and display the same entry and resulting balances.

README.md:38,48,88,110 · crates/ledger-recovery/src/journal.rs:100 · crates/ledger-node/src/main.rs:36 · crates/ledger-api/proto/ledger/v1/ledger.proto:5

Resolve a timeout, an overload or a partial hold

Same-ID retries answer the most important lost-reply question. A backend can query an original receipt or a known entry. Temporary queue saturation and permanent size/shape limits share RESOURCE_EXHAUSTED. The SDK exposes an initial refusal, while the benchmark retries it. A structured reason and bounded same-ID retry example would let integrators choose the right response.

After a partial capture, the public API cannot return the current remaining reservation lines by reservation ID. It also lacks the embedded interface's consistent multi-account lookup. Prioritize the first for settlement support and the second if clients reconcile multiple balances at one point.

crates/ledger-node/src/client_service.rs:70,113 · ledger-api/src/client.rs:227 · ledger-api/proto/ledger/v1/ledger.proto:9 · ledger-core/src/reservation.rs:22 (under crates/).

Operate a running cluster

The runbook asks an operator to wait for a restarted voter to catch up before stopping another. NodeStatus exposes peer progress, term, applied index, protected index and recent errors inside the process. The standalone CLI and business API do not expose that live view. Add an authenticated operator status command with a clearly defined "caught up" condition. Distinguish a reachable listener from a leader able to serve protected requests.

The dedicated backend CA establishes one trust group. Every admitted backend can change account bounds, and the stored decision contains the request, command and result without an authenticated caller identity. A single trusted integration can accept this model. Separate team permissions and caller attribution become important when trust roles diverge.

README.md:86,155 · crates/ledger-node/src/host.rs:57 · crates/ledger-replica/src/replica.rs:301 · crates/ledger-core/src/engine.rs:14

Throughput, latency and overload

Public results use three voters, a shared local journal, eight warmed TLS connections and 32 closed-loop callers. A new payment decision includes a durable business rejection. Reads and duplicate calls are counted separately.

Workload	Nov 8 decisions/s	Nov 9 repeats decisions/s	Nov 8 p99 ms	Nov 9 repeats p99 ms
Distributed wallets	847.9	787.3 / 842.2	41.22	45.30 / 36.90
One contended wallet	850.2	714.1 / 893.7	38.29	139.91 / 36.75
One follower offline	819.3	786.7 / 887.6	47.39	36.33 / 38.57
Follower returning	826.4	544.1 / 841.2	45.33	125.02 / 45.06

Nov 8 figures are medians of three per-run statistics. Nov 9 pairs show both complete repeats in run order with batch diagnostics enabled. In each contended phase 1,792 of 2,048 payments posted and 256 reached the configured floor. The catch-up phase spans the entire foreground workload, including time after the voter caught up.

Fixed arrivals reveal pressure that a closed loop hides

At **300 mixed operations/s for 60 s**, the first complete repeat finished **17,692 of 18,000**. The sender skipped 308 before transmission at its 64-outstanding limit. All submitted operations finished. Across those submissions 9,139 (51.7%) saw a first-attempt refusal. Payment p50 / p95 / p99 were **54.0 / 86.8 / 207.8 ms**. The harness included retry delay and reconciled 21,205 unique posted payments across all phases against 65 accounts.

The prior 300/s trials completed all 18,000, with payment p99 of 59.6 and 86.2 ms. At 3,000 offered/s in the older 4-second pressure test, only 7,142 of 12,000 scheduled operations reached completion. The sender dropped 40.5% before transmission. Quoting its submitted-payment p99 of 73.13 ms alone would conceal that missing demand.

The second complete repeat finished 18,000 of 18,000 scheduled operations, with 0 sender drops and a submitted-payment p99 of 77.26 ms.

Where batching pays

With 24 embedded callers and the same 384 decisions, allowing 256 commands per entry raised throughput from **151.6 to 583.5 decisions/s** and reduced p99 from 1,137 to 50.9 ms. The two runs posted 360 and rejected 24. A single caller saw 34.3 versus 37.5 decisions/s. The ordering was fixed and the one-command case incurred 9,451 admission refusals. Batching, contention and retry time all contribute to the difference.

docs/measurements/2026-11-08-public-load.json · docs/measurements/2026-11-08-local-load.md:9,24,29 · crates/ledger-node/tests/network/api/load.rs:147,393 · group_commit.rs:8. Attached November 9 release logs.

Recovery cost and scale limits

The project has functioning restore, scrub and reclamation tools. The expensive paths are conditional on disk state, usable checkpoints and the amount of missing history.

Scenario	Evidence	Operational meaning
All voter directories lost	Recorded at 65,538 original outcomes: 10.265 s from fence to activation from genesis, 7.776 s with a base.	Local dev profile. Checkpoint construction, restart, election and first protected payment are outside the recovery interval.
Returning follower, different SST layout	Fresh release drill: 251,606 bytes versus 5,648,418 estimated for physical transfer on the same offer. Catch-up after reopen took 1.12 s.	Replay saved 95.55% of payload. Concurrent writes had 83.81 ms p99 against 48.09 ms in the matched foreground control.
Decoded account cache	Recorded 16-account hot set: 100% hits, 0.334 µs local lookup p99. Wide traversal: zero hits, 27.917 µs p99.	This is a local replica lookup microbenchmark. End-to-end protected reads still pay for quorum and journal confirmation.

Source finding / Large snapshot transfers can lose progress

The physical path requests one 256 KiB chunk and syncs it before fetching the next. A cold 1 TB decimal image at 25 ms round trip needs at least 26.5 hours for these sequential requests alone. An offer expires at 24 hours. If the next offer names a newer seal, prior staged files are released. One incoming slot can also be held by an old leader’s valid transfer until it ends. Test a slow large transfer across a leader change and newer checkpoint before setting a recovery target.

crates/ledger-node/src/transport/snapshots.rs:26 · transport/snapshots/receive.rs:149,258 · crates/ledger-replica/src/checkpoint_files/transfer.rs:12,211,422 · ledger-replica/src/raft_store.rs:517.

Source finding / Restoring a voter does not repair a damaged base

A checkpoint refresh scrubs the existing independent image or opens it as the starting state. If that image is damaged, full journal replay can restore each voter, but leaves the published base in place. Qualify a supported rebuild from the verified full journal with existing archive pins preserved. A separate interrupted restore can leave a full staging directory beside the destination, so retry also needs a disk-space budget.

crates/ledger-replica/src/recovery/base.rs:70,172 · crates/ledger-replica/src/recovery.rs:148,194,501 · crates/ledger-node/tests/recovery_admin.rs:506

Filesystem assumptions matter. A separate directory on the same computer exercises the journal protocol. Protecting payments through host loss also requires independent storage with verified locking, rename and sync behavior. The production volume, long-duration growth, cross-host latency and terabyte-scale behavior remain unmeasured.

docs/measurements/2026-11-08-local-load.md:48,65 · docs/rfcs/0010-account-cache.md:27 · docs/rfcs/0015-offsite-commit-protection.md:23

Release gates and evidence

These are proposed acceptance tests in priority order. Set acceptable error, latency and recovery targets with the intended users before declaring capacity.

Priority	Work	Acceptance evidence
1 / Reliability	Instrument journal lock waits, hold time and operation type. Exercise follower restart and permanent disk loss.	A returning voter rejoins during sustained writes. From a blank replacement, measure time to regain a third voter and any pause in protected service.
2 / Operations	Ship authenticated live voter status and rehearse the deployment journal on independent storage.	An operator can verify the surviving majority, journal coverage and restored redundancy. A full disaster drill recovers every acknowledged receipt and measures time to the first payment.
3 / Capacity	Test target hardware, networking and storage with warm and cold clients, a representative transfer mix and active maintenance.	Report offered, submitted, refused, retried and completed counts alongside payment/read p99, peak disk use and catch-up time. Run through checkpoint and compaction cycles.
4 / Integration	Provide a first-payment walkthrough, machine-readable refusal reasons and a protected reservation lookup.	A new backend team can submit and reconcile a payment, recover a lost reply and state the remainder of a partially captured hold.
5 / Regression	Automate workspace checks and retain replay fixtures across release versions. Check concurrent response histories under recorded fault schedules.	Old requests retain the same receipt and effects across releases. A repeatable test checks real-time ordering during simultaneous failover, retries and journal delays.

Verification record

Executed November 9. Locked workspace tests passed (216, including one documentation test). The two explicitly ignored default-network measurements were run separately in release mode with batch diagnostics, alongside the feature-gated group-commit comparison. Workspace formatting passed. Strict Clippy checks passed across all targets and features. CLI help and journal setup were exercised using the README configuration in a temporary directory.

Reproduce. Use the documented native build requirements and protoc 3.21.12. Run the commands below. Repeat the public load case independently with the same environment and fresh cluster directories.

```
cargo test --locked --workspace --no-fail-fast
LEDGER_LOAD_RATE=300 LEDGER_LOAD_SECONDS=60 cargo test --release \
--locked -p ledger-node --features batch-diagnostics --test network \
-- --ignored --nocapture --test-threads=1
```

Scope. The review used the local workspace snapshot with no Git revision metadata, Rust 1.99.1 and macOS 27.2 arm64. The checked-in measurements describe their own November 8 host. Four run transcripts are attached inside this PDF. No value for a production SLO or multi-host failure rate can be inferred from these trials.

Primary references: README.md · ARCHITECTURE.md · docs/rfcs/0001–0017 · docs/measurements/2026-11-08-{public-load,snapshot-replay}.json · Raft timing model, Ongaro & Ousterhout (2014), §5.6.